Student Name: Faraz Sahabi

Student ID: 34557

CS405 Project 1 Report

Painting chosen: Feyhaman Duran — Kabataş Rıhtımı (1936)

Why: its perspective cues (the long diagonal of the jetty, the quay in the foreground) suit the camera work of Part C, and the hull of the sailing boat is a clean, smooth contour that a three-segment Bézier curve can trace in Part D.

Frame Times: measured in Chrome with the main view and the curve animation running 16 ms per frame (60 fps), CPU time about 1.0 ms, 13 draw calls, 7,302 vertices per frame.

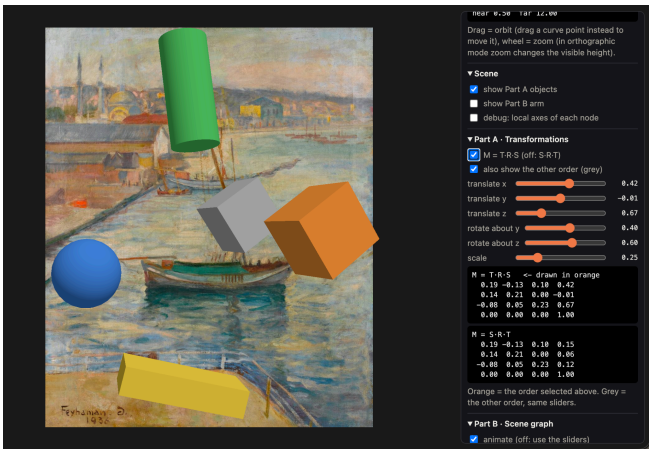
GPU: Apple M1 integrated GPU (Metal 3)

Repository: https://github.com/FarazSahabi/cs405-project1.git

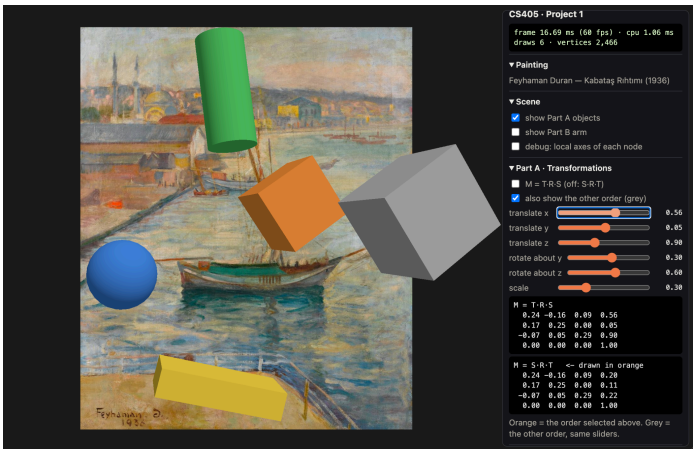
Part A: Transformations

The library implements identity, multiply, translate, rotateX/Y/Z, scale, transpose and the rigid inverse (R^T and $-R^Tt$). Three objects stand fixed in front of the painting; the orange box is driven by sliders (translation, rotation about y and z, uniform scale); a switch chooses $M = T \cdot R \cdot S$ or $M = S \cdot R \cdot T$, a grey box shows the other order, and both matrices are printed.

With uniform scale s, $T \cdot R \cdot S$ gives $p' = s \cdot R \cdot p + t$: the box is scaled, then rotated, then placed at t, so the translation is exactly what the slider says. $S \cdot R \cdot T$ gives $p' = s \cdot R \cdot (p + t) = s \cdot R \cdot p + s \cdot R \cdot t$: the translation itself is rotated and scaled, so the box lands somewhere else and moves along a rotated path when the sliders change. Example: p = (1,0,0), t = (1,0,0), R = Rz(90°), s = 2. $T \cdot R \cdot S$: S p = (2,0,0), R → (0,2,0), T → (1,2,0). $S \cdot R \cdot T$: T p = (2,0,0), R → (0,2,0), S → (0,4,0). The rest of the code uses $T \cdot R \cdot S$, because every scene-graph local matrix is built as translation · rotation, with sizes kept in a separate visual matrix.



M = T·R·S (orange): matrices printed in the panel.



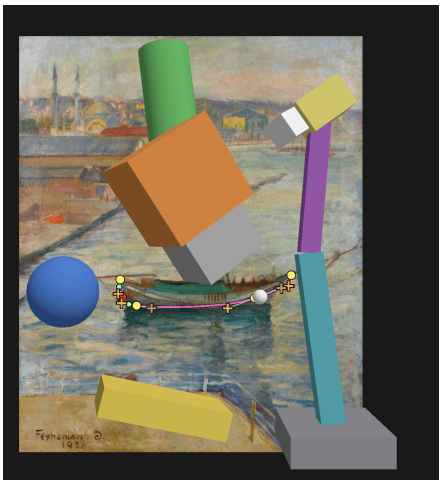
M = S·R·T (orange) with the grey box showing T·R·S.

Part B: Scene graph

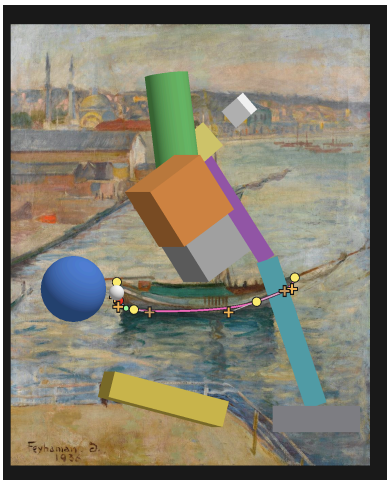
A node stores a local matrix, a parent and children; updateWorld(parentWorld) sets world = parent.world · local and recurses, so one call from the root refreshes the tree. The arm has 6 nodes in 6 levels (the root is level 0). The three joint angles and the satellite's orbit are animated; a debug option draws every node's local axes (x red, y green, z blue). Sizes are applied by a separate visual matrix when drawing (draw matrix = world · visual), so a bar's length never scales its children.

Part C: Camera and projection

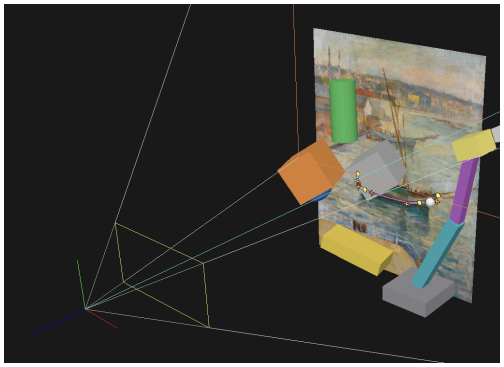
The view matrix puts the camera at the origin looking down −z. From the eye, target and up vector I compute f = normalise(target − eye), r = normalise(f × up) and u = r × f (two cross products). The rotation block of the view matrix has the rows r, u, −f, and its translation is −R·eye. If up is parallel to f, the code falls back to another up vector. The perspective matrix scales x by t/aspect and y by t, with t = 1/tan(fovY/2), and puts −z into w. For WebGPU's depth range [0, 1] I wrote z_clip = A·z + B and solved for depth 0 at z = −n and 1 at z = −f. This gives A = f/(n − f) and B = n·f/(n − f). The orthographic matrix maps depth linearly with z′ = (z + n)/(n − f), which behaves the same at the near and far planes. A second, fixed inspection camera draws the main camera's frustum, with its corners built in view space and mapped to world space by the rigid inverse of the view matrix. I chose near = 0.5 and far = 12, because the scene lies about 1.6 to 3.2 units from the camera, so nothing is clipped. Depth is stored non-linearly, and its resolution worsens with the square of the distance and improves as the near plane grows. At a distance of 2.8, near = 0.5 resolves about $2.3 \cdot 10^{-4}$ units in a 16-bit buffer, while near = 0.01 and far = 100 resolves only about $1.2 \cdot 10^{-2}$, which would cause z-fighting. The renderer uses a 24-bit buffer, which copes with either choice, but the larger near plane is safer.



Main camera (perspective)



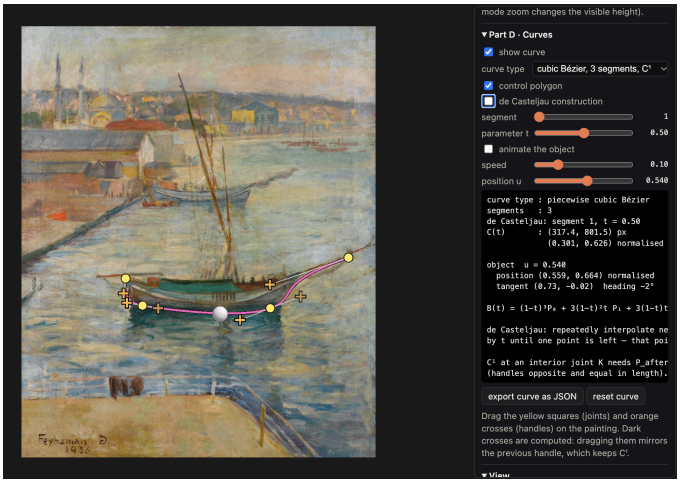
Main camera (orthographic)



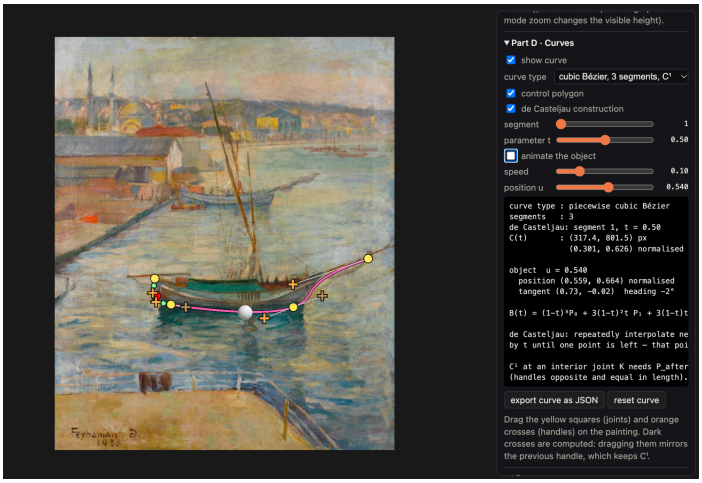
Inspection Camera

Part D: Curves over the painting

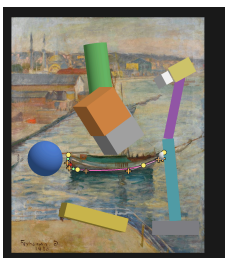
A cubic Bézier curve is $B(t) = \Sigma\, B_i(t) \cdot P_i$ with the Bernstein weights $B_0 = (1-t)^3$, $B_1 = 3(1-t)^2 t$, $B_2 = 3(1-t)t^2$ and $B_3 = t^3$. They are non-negative and sum to 1, so the curve stays inside the convex hull of its control points. At $t = 0$ only B_0 is non-zero, so $B(0) = P_0$, and at $t = 1$ only B_3 is, so $B(1) = P_3$. The derivative is $B'(0) = 3(P_1 - P_0)$ and $B'(1) = 3(P_3 - P_2)$. For two segments to be C^1 at their shared joint K, the derivatives $3(K - P_2)$ and $3(P_1' - K)$ must be equal, which gives $P_1' = 2K - P_2$. The code enforces this by computing the first handle of each later segment this way, so a joint stays C^1 however you drag. de Casteljau's construction interpolates neighbouring points by t until one point is left, and that point is B(t). I trace the boat's hull with three such segments and the jetty edge with a Catmull–Rom spline. The mouse ray is intersected with the painting's plane, so control points are dragged in the picture, and the curve is exported in normalised coordinates. A ball travels along the curve and its heading follows the tangent.



The Bézier curve



de Casteljau construction



JSON Appendix


```
{ "painting": "kabatas",
 "kind": "bezier-c1",
"segments": [
        [ [ 0.2957, 0.5844 ], [ 0.29, 0.6188 ], [ 0.2995, 0.6406 ], [ 0.3431, 0.6469 ] ],
        [ [ 0.3431, 0.6469 ], [ 0.3867, 0.6531 ], [ 0.6066, 0.6531 ], [ 0.6844, 0.6289 ] ],
        [ [ 0.6844, 0.6289 ], [ 0.7621, 0.6047 ], [ 0.7829, 0.6 ], [ 0.7905, 0.5742 ] ] ]
}
```


What Went Wrong and How I Found it

1. **A stale file: “**npm test” still showed lookAt, perspective and orthographic as "TODO", with the same stack trace as before. That meant my copy of mat4.js was out of date. Comparing file sizes (6003 against 7778 bytes) showed it, and replacing the file fixed it.
2. **The arm hidden under the panel:** the arm stood at a fixed x = 1.0, but the painting is portrait, so the arm was partly outside the picture and under the side panel. A screenshot showed it. I now place the objects relative to the painting's width and added sliders that move the arm's base.
3. **Lines too thin to see:** one-pixel lines for the curve and control points were nearly invisible on the painted texture. I rebuilt them as thick triangle strips with a dark outline and filled markers.

AI Appendix

I used Claude AI as a coding assistant for debugging and assisting through complex coding syntax I was familiar with on a low level throughout this project. It also explained each part to me before and after writing it.

Note: Bonus also completed